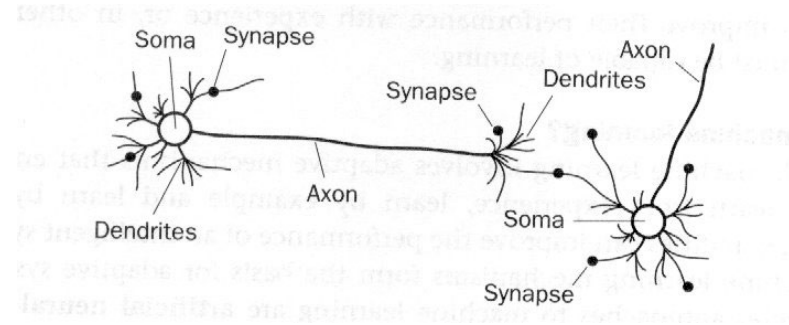


Class 02A: Model of an Artificial Neural Network

A neural network is a computational model inspired by the structure and function of the human brain. Imagine how you learn to recognize a friend's face.



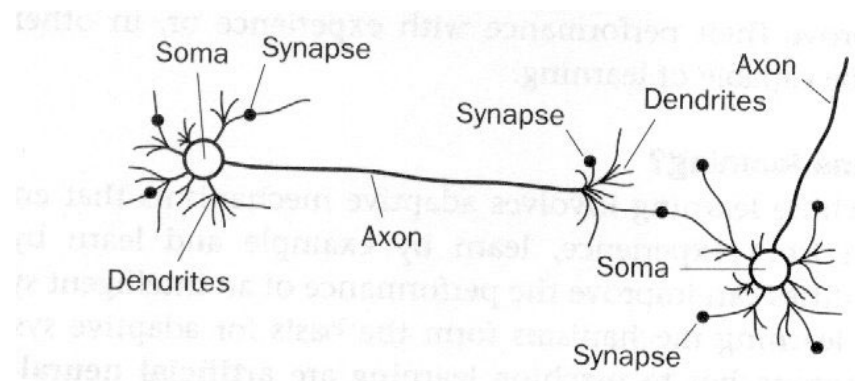
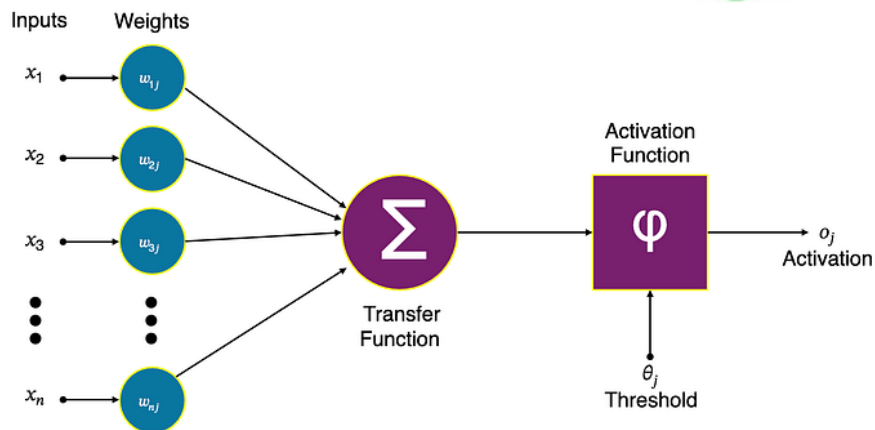
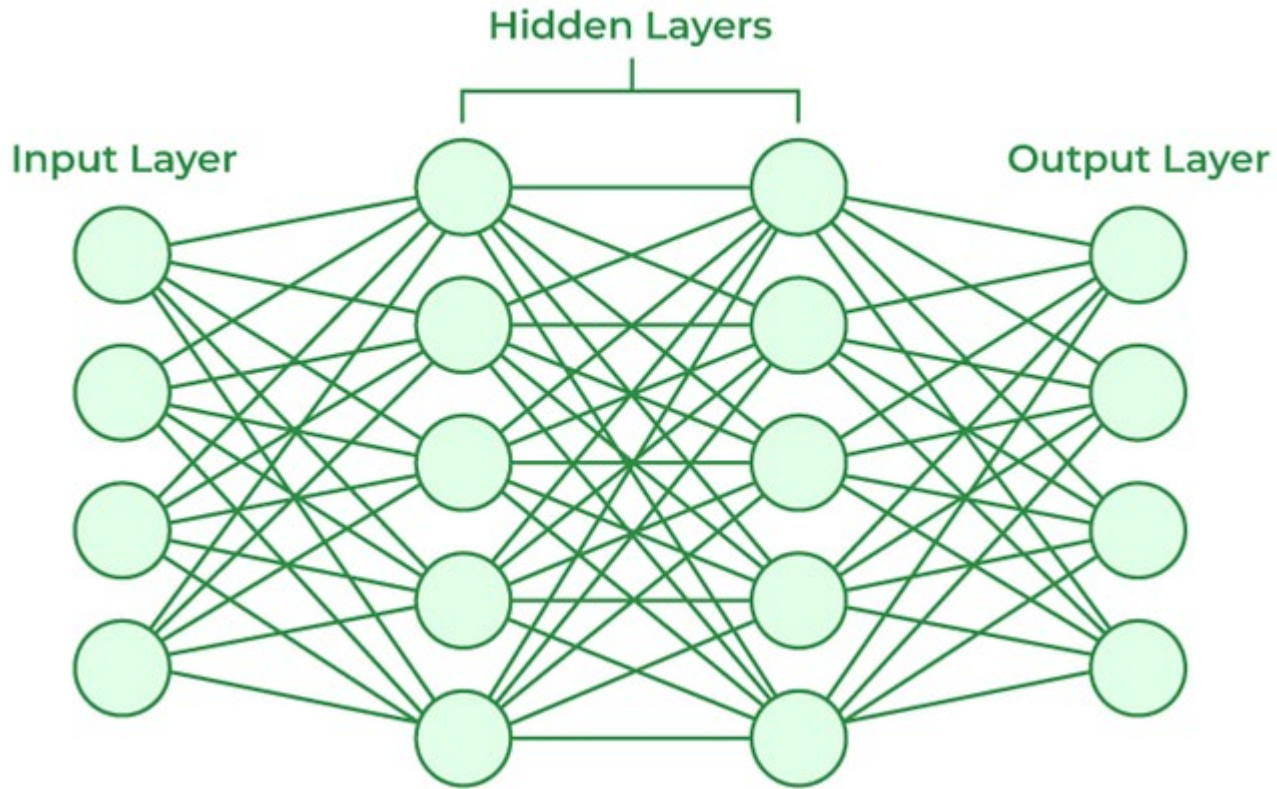
You don't consciously measure the distance between their eyes or the curve of their smile. **Your brain has been trained on countless examples, allowing it to automatically recognize a complex pattern of features.**

Neural networks work in a similar way.

They are good at tasks that are intuitive for humans but have proven difficult for traditional programming, such as recognizing speech, translating languages, or identifying objects in images.

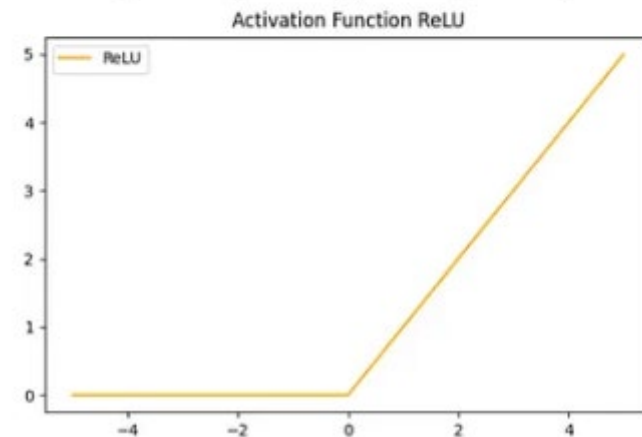
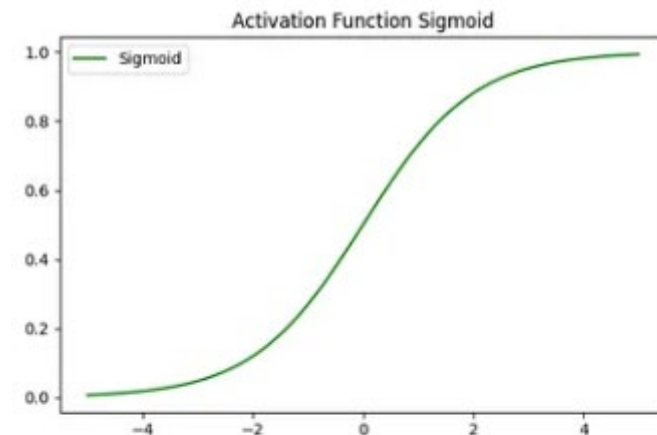
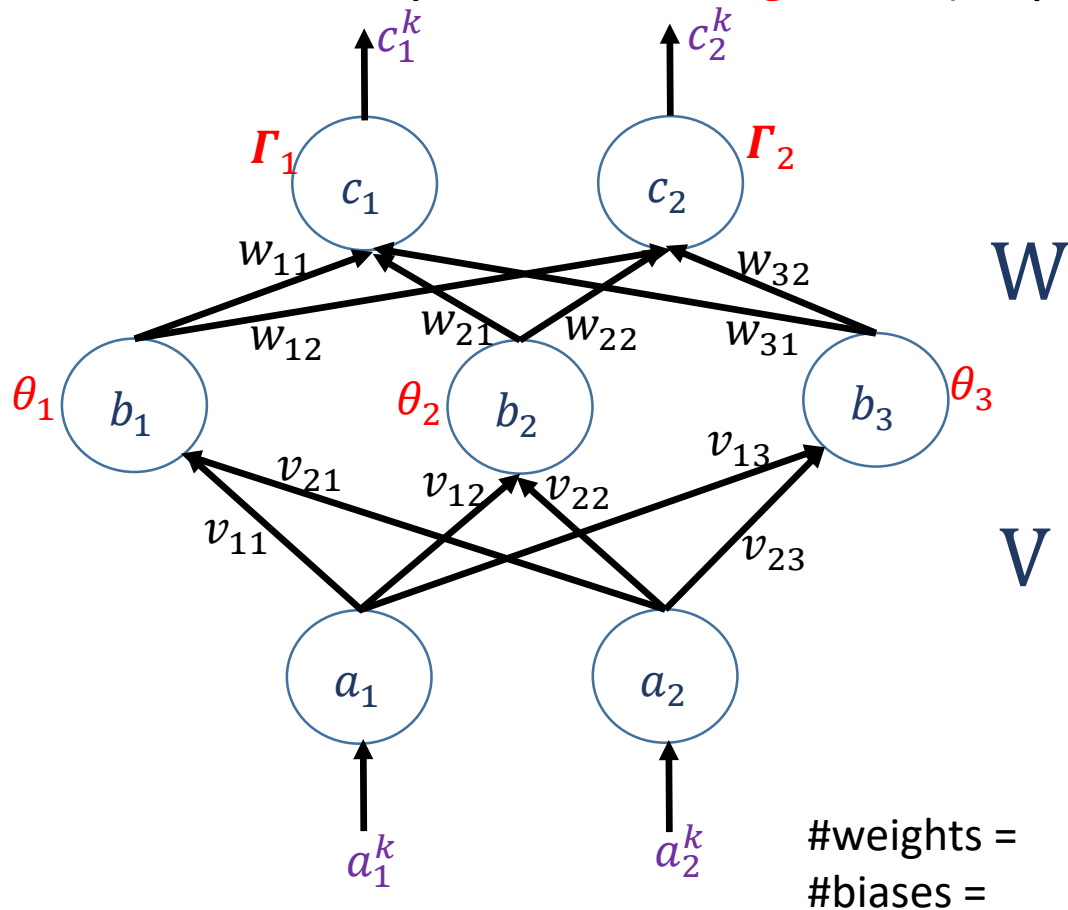


A simple neural network architecture with input, hidden, and output layers.

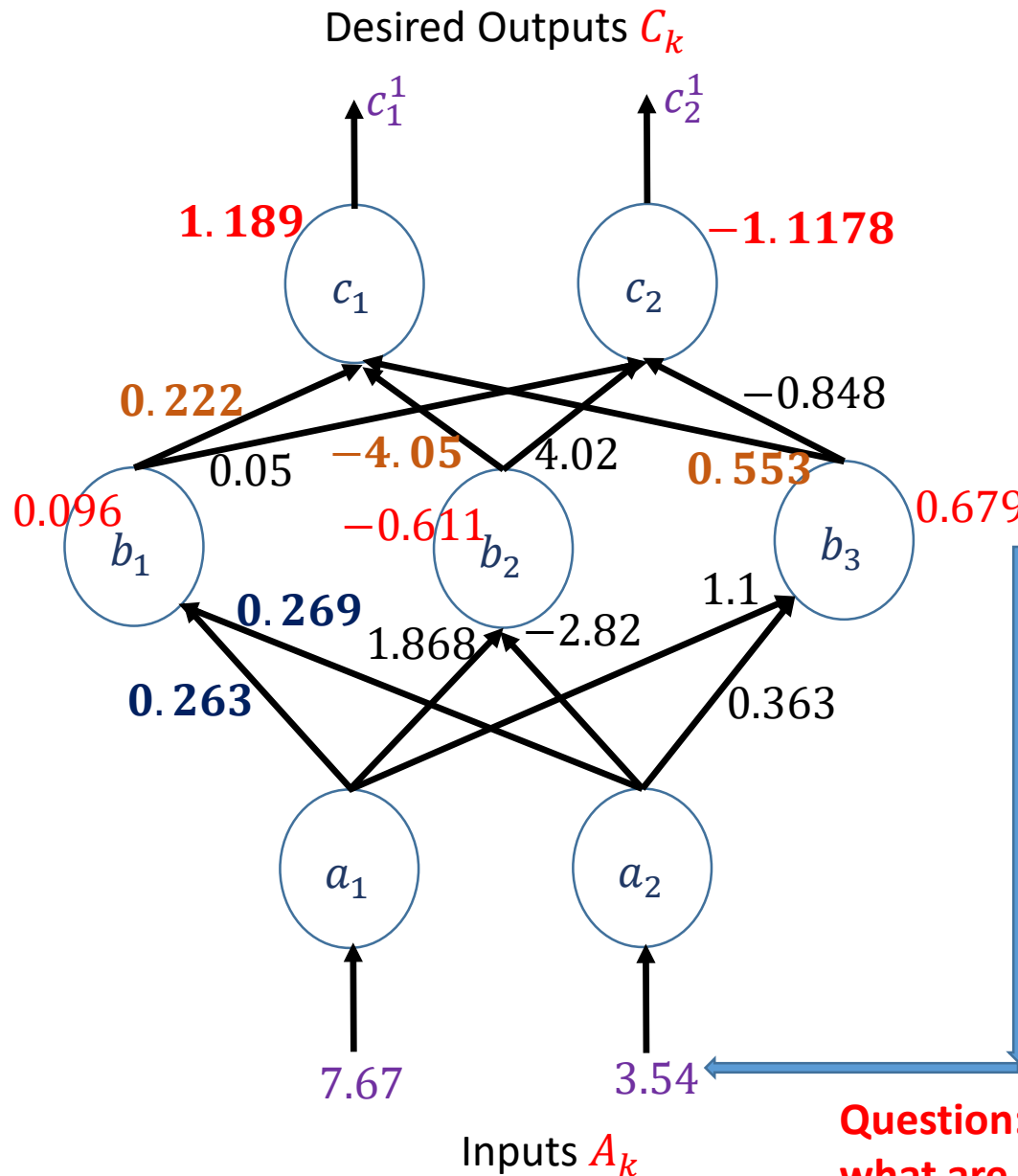


Maths of the Model of an Artificial Neural Network

The following provides an example of a two-input, two-output ANN with one hidden layer. The ANN has three *sigmoid* function hidden units, and the outputs can be *sigmoid* (or purelin) function.



What are the values of the two outputs ?



Given training data:

Inputs A_k		outputs C_k	
a_1^k	a_2^k	c_1^k	c_2^k
7.67	3.54	...	...
a_1^2	a_2^2	c_1^2	c_2^2
...	...	...	...

Question: If the two inputs are 7.67 and 3.54, what are the two outputs ?

Answer of Feed-forward:

```
# Define the activation functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

$$a_1 = 7.67 ; a_2 = 3.54$$

$$k1 = (0.263)(7.67) + (0.269)(3.54) + 0.096 = 2.01 + 0.95 + 0.096 = 3.067$$

$$b_1 = \frac{1}{1+e^{-k1}} = \frac{1}{1+e^{-3.067}} = 0.955$$

$$k2 = (1.868)(7.67) + (-2.82)(3.54) - 0.611 = 14.33 - 9.983 - 0.611 = 3.732$$

$$b_2 = \frac{1}{1+e^{-k2}} = \frac{1}{1+e^{-3.732}} = 0.9765$$

$$k3 = (1.1)(7.67) + (0.363)(3.54) + 0.679 = 8.437 + 1.285 + 0.679 = 10.44$$

$$b_3 = \frac{1}{1+e^{-k3}} = \frac{1}{1+e^{-10.44}} = 0.999$$

$$h1 = (b_1)(0.222) + (b_2)(-4.05) + (b_3)(0.553) + 1.189 = -2.001$$

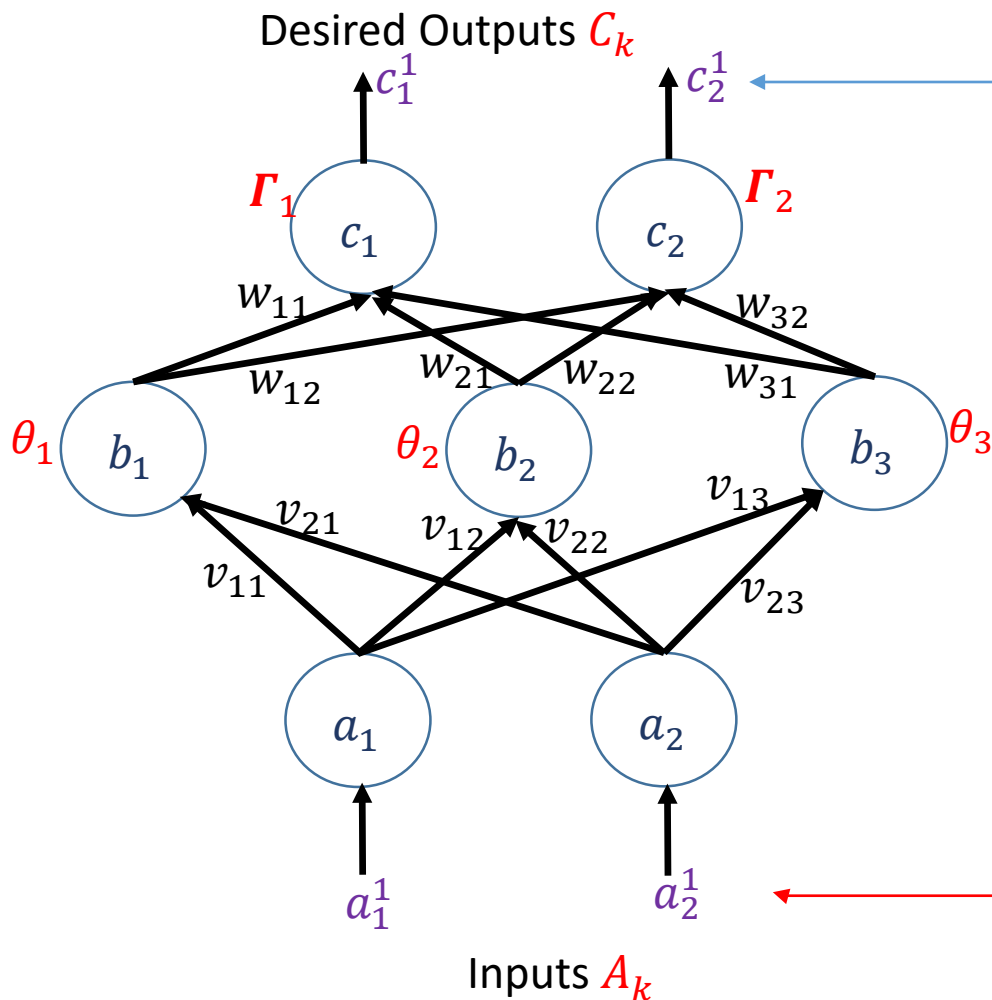
$$c_1 = \frac{1}{1+e^{-h1}} = \frac{1}{1+e^{2.001}} = 0.1191 \quad (\text{note that output is a sigmoid function})$$

Similarly, $c_2 = ?$ (obtain that value as an exercise)

Answer: $c_2 = 0.8817$

What is the issue of **training / learning** ?

Given the architecture of the ANN (two-input, two-output, three sigmoid function hidden units), as well as training data, **find the weights and biases**.



Given training data:

Inputs A_k		outputs C_k	
a_1^k	a_2^k	c_1^k	c_2^k
a_1^1	a_2^1	c_1^1	c_2^1
a_1^2	a_2^2	c_1^2	c_2^2
...	...	...	...

Regression Problem:
The goal is to minimize the loss (Mean Square Error, MSE).

The Backpropagation Algorithm (BP)

1. Assign random values in the range $[-1,1]$ to v_{ij} and w_{ij} as well as θ_i , Γ_i .

2. Training data is a pattern pair (A_k, C_k) , $k = 1, \dots, m$.

3. Take the first training pair of data ($k=1$),

(a) Feedforward from input to hidden layer:

$$b_1 = f(a_1 v_{11} + a_2 v_{21} + \theta_1),$$

and f is the logistic sigmoid function. Similarly,

$$b_2 = f(a_1 v_{12} + a_2 v_{22} + \theta_2),$$

$$b_3 = f(a_1 v_{13} + a_2 v_{23} + \theta_3),$$

(b) Feedforward from hidden to output layer:

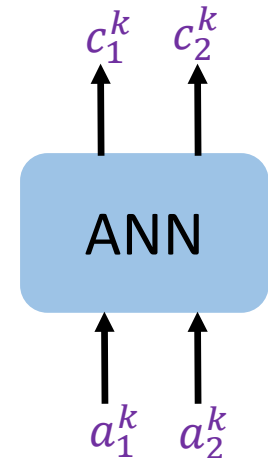
If the output is a sigmoid unit, then

$$\begin{aligned} c_1 &= f(b_1 w_{11} + b_2 w_{21} + b_3 w_{31} + \Gamma_1) \\ c_2 &= f(b_1 w_{12} + b_2 w_{22} + b_3 w_{32} + \Gamma_2) \end{aligned}$$

If the output is a purelin unit:

$$c_1 = (b_1 w_{11} + b_2 w_{21} + b_3 w_{31} + \Gamma_1)$$

$$c_2 = (b_1 w_{12} + b_2 w_{22} + b_3 w_{32} + \Gamma_2)$$



The Backpropagation Algorithm

(c) Calculate the error at the output layer:

If the output is a sigmoid unit, then

$$d_1 = (c_1^k - c_1) c_1 (1 - c_1)$$
$$d_2 = (c_2^k - c_2) c_2 (1 - c_2)$$

If the output is a purelin unit:

$$d_1 = c_1^k - c_1 \quad (\text{desired_output} - \text{output})$$
$$d_2 = c_2^k - c_2 \quad (\text{desired_output} - \text{output})$$

(d) Calculate the error at the hidden layer:

$$e_1 = b_1 (1 - b_1) [w_{11} d_1 + w_{12} d_2]$$
$$e_2 = b_2 (1 - b_2) [w_{21} d_1 + w_{22} d_2]$$
$$e_3 = b_3 (1 - b_3) [w_{31} d_1 + w_{32} d_2]$$

(e) Adjust all weights between the hidden and output layer:

$$w_{11}^{new} = w_{11}^{old} + \alpha b_1 d_1$$
$$w_{12}^{new} = w_{12}^{old} + \alpha b_1 d_2$$
$$w_{21}^{new} = w_{21}^{old} + \alpha b_2 d_1$$
$$w_{22}^{new} = w_{22}^{old} + \alpha b_2 d_2$$
$$w_{31}^{new} = w_{31}^{old} + \alpha b_3 d_1$$
$$w_{32}^{new} = w_{32}^{old} + \alpha b_3 d_2$$

(f) **Adjust the biases** at the output units:

$$\Gamma_1^{new} = \Gamma_1^{old} + \alpha d_1$$

$$\Gamma_2^{new} = \Gamma_2^{old} + \alpha d_2$$

(g) **Adjust all weights** between and the input and hidden layer:

$$v_{11}^{new} = v_{11}^{old} + \beta a_1 e_1$$

$$v_{21}^{new} = v_{21}^{old} + \beta a_2 e_1$$

$$v_{12}^{new} = v_{12}^{old} + \beta a_1 e_2$$

$$v_{22}^{new} = v_{22}^{old} + \beta a_2 e_2$$

$$v_{13}^{new} = v_{13}^{old} + \beta a_1 e_3$$

$$v_{23}^{new} = v_{23}^{old} + \beta a_2 e_3$$

(e) **Adjust the biases** at the hidden units:

$$\theta_1^{new} = \theta_1^{old} + \beta e_1$$

$$\theta_2^{new} = \theta_2^{old} + \beta e_2$$

$$\theta_3^{new} = \theta_3^{old} + \beta e_3$$

4. Repeat the adjustment to all 12 weights and 5 biases for all training pairs for $k = 1, \dots, m$.

5. After one **epoch**, obtain the max error and average sum of errors. Stop if the average error is very small.

Two important things you should know before studying the proof

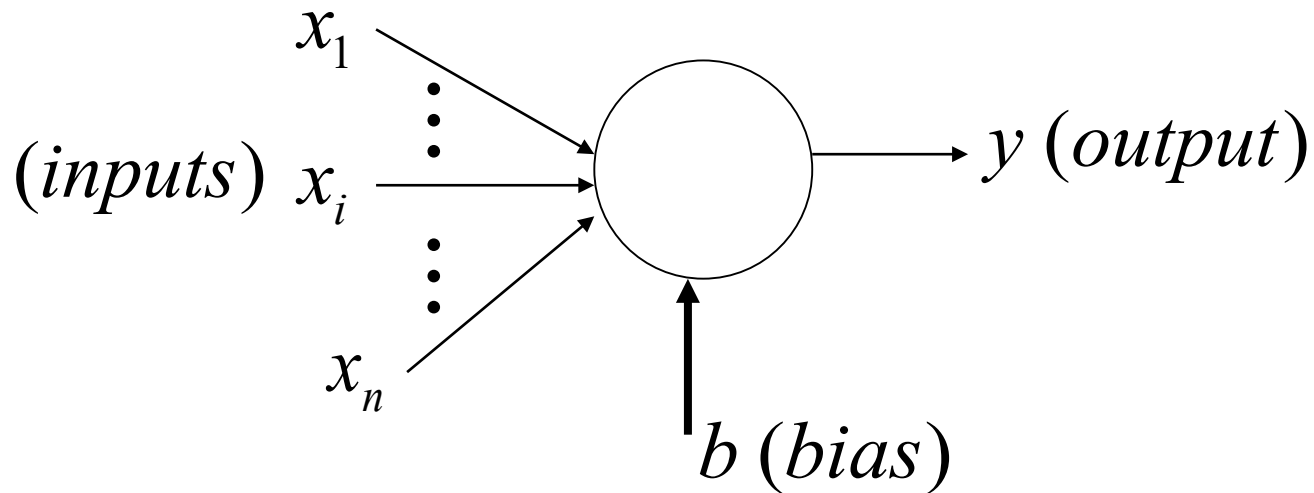
- Derivative of the sigmoid function (f)

$$y = f(a); \quad y' = f'(a) = f(a)(1 - f(a))$$

- Gradient descent-based method : to minimize a function $E(x)$

$$x^1 = x^o - \alpha \frac{dE(x^o)}{dx}$$

Sigmoid function (f) and its derivative



$$y = f\left(\sum_{i=1}^n x_i + b\right)$$

where $f(a) = \frac{1}{1 + e^{-a}} = \boxed{(1 + e^{-a})^{-1}}$

Derivative of the sigmoid function

$$\begin{aligned} f'(a) &= (-1)(1 + e^{-a})^{-2} e^{-a} (-1) = \frac{e^{-a}}{(1 + e^{-a})^2} = \frac{1}{1 + e^{-a}} \cdot \frac{e^{-a}}{1 + e^{-a}} \\ &= \frac{1}{1 + e^{-a}} \cdot \frac{1 + e^{-a} - 1}{1 + e^{-a}} = f(a)(1 - f(a)) \end{aligned}$$

- **The sigmoid function is popular & expressible.**
- **Its derivative with respect to the function variable is very easy to form.**

Gradient Descent-based Method

Given a function $E(x)$, we may minimize the function by calculating $\frac{dE(x)}{dx}$ (gradient of E) and adjusting x until we reach a minimum. The algorithm for the minimization of a function is called the steepest descent.

Procedures:

1. Make an initial guess x^0 .

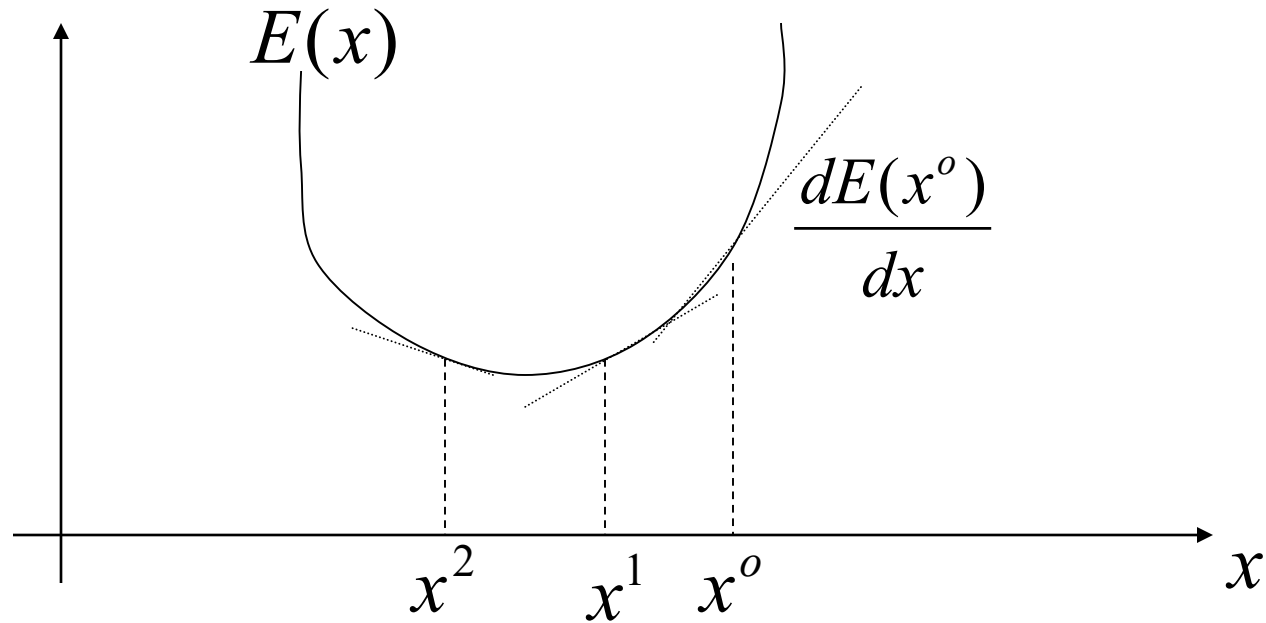
2. Compute the gradient $\frac{dE(x)}{dx}$ at x^0 i.e. $\frac{dE(x^0)}{dx}$

3. Adjust x^0 to *obtain* x^1 by moving in a direction opposite to the gradient i.e.

$$x^1 = x^0 - \alpha \frac{dE(x^0)}{dx}$$

4. Stop when $(x^1 - x^0)$ is sufficiently small. The parameter α controls the adjustment.

Graphical explanation



$$x^1 = x^o - \alpha \frac{dE(x^o)}{dx}$$

$$x^2 = x^1 - \alpha \frac{dE(x^1)}{dx}$$

$$x^3 = x^2 - \alpha \frac{dE(x^2)}{dx}$$

NOTE :

α is a parameter to control the rate of convergence. If α is too large, the value of x may not converge.

Extension to higher-dimension of Gradient Descent-based Method

$$\underline{x} = \begin{pmatrix} x_a \\ x_b \end{pmatrix} ; \frac{dE(\underline{x})}{d\underline{x}} = \begin{pmatrix} \frac{\partial E(\underline{x})}{\partial x_a} \\ \frac{\partial E(\underline{x})}{\partial x_b} \end{pmatrix}$$

Adjust $\underline{x}^o$ to *obtain* $\underline{x}^1$ by moving in a direction opposite to the gradient i.e.

$$\underline{x}^1 = \underline{x}^o - \alpha \frac{dE(\underline{x}^o)}{d\underline{x}}$$

$$\begin{pmatrix} x_a^1 \\ x_b^1 \end{pmatrix} = \begin{pmatrix} x_a^o \\ x_b^o \end{pmatrix} - \alpha \begin{pmatrix} \frac{\partial E(\underline{x}^o)}{\partial x_a} \\ \frac{\partial E(\underline{x}^o)}{\partial x_b} \end{pmatrix}$$

Hence,

$$x_a^1 = x_a^o - \alpha \frac{\partial E(\underline{x}^o)}{\partial x_a}$$

$$x_b^1 = x_b^o - \alpha \frac{\partial E(\underline{x}^o)}{\partial x_b}$$

Example of Gradient Descent-based Method

$$E(\underline{x}) = 5x_a^2 + 2x_ax_b + 2x_b^2 - 14x_a - 10x_b + 17$$

$$\frac{\partial E(\underline{x})}{\partial x_a} = 10x_a + 2x_b - 14 \quad ; \quad \frac{\partial E(\underline{x})}{\partial x_b} = 2x_a + 4x_b - 10$$

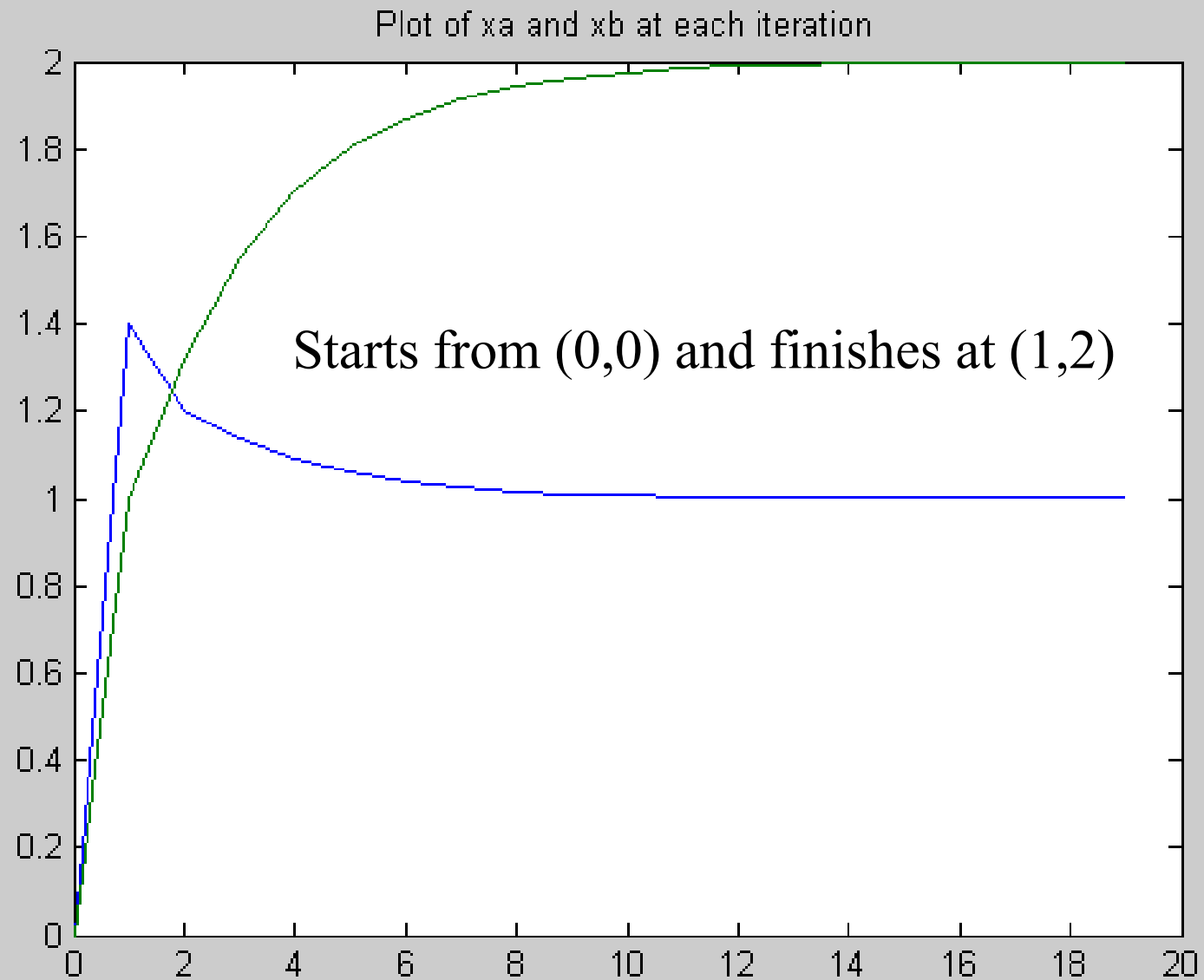
$$x_a^1 = x_a^0 - \alpha \frac{\partial E(\underline{x}^o)}{\partial x_a}$$

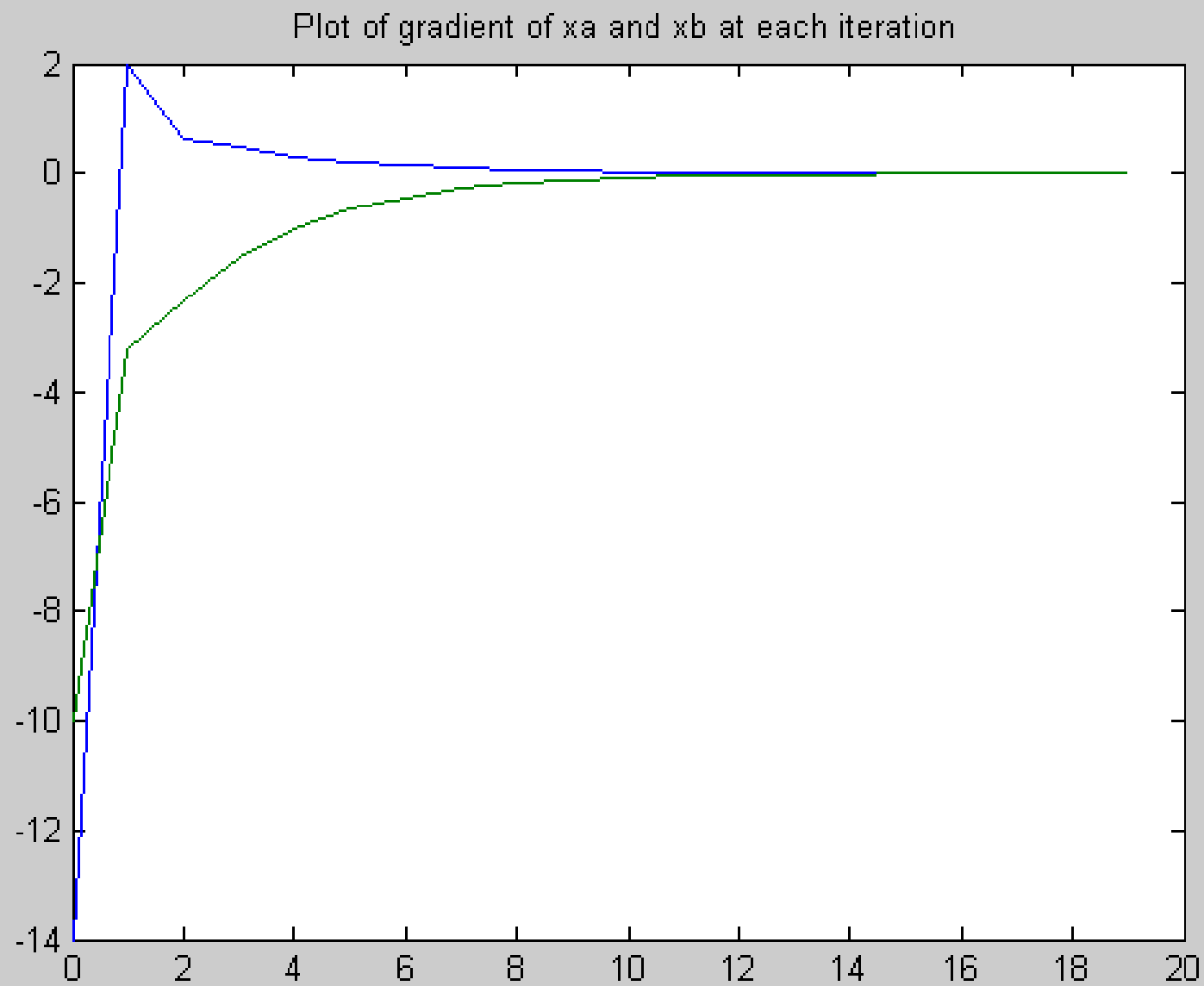
$$x_b^1 = x_b^0 - \alpha \frac{\partial E(\underline{x}^o)}{\partial x_b}$$

Pseudo code implementation

```
% Filename: gradient.txt
% Demo of gradient descent-based search procedure
% define n,xa,xb,K
% exact solution at xa=1,xb=2
clear all, close all
n=20;xa=0;xb=0;K=0.1; % stable
% n=20;xa=0;xb=0;K=0.2; % unstable
% n=20;xa=5;xb=7;K=0.1; % stable
plotxa(n,1)=0; plotxb(n,1)=0; plotdxa(n,1)=0; plotdxb(n,1)=0; % initialization

for i = 1:n
    plotxa(i)=xa; % for plotting
    plotxb(i)=xb;
    dEdxa=10*xa+2*xb-14;
    dEdxb=2*xa+4*xb-10;
    xa=xa-K*dEdxa;
    xb=xb-K*dEdxb;
    plotdxa(i)=dEdxa; % for plotting
    plotdxb(i)=dEdxb;
end
figure, plot((0:n-1)',[plotxa plotxb])
title('Plot of xa and xb at each iteration')
figure, plot((0:n-1)',[plotdxa plotdxb])
title('Plot of gradient of xa and xb at each iteration'), figure
```





The proof for the BP equations:

Let E be the error function that we would like to minimize.

$$E = \frac{1}{2} [(c_1^k - c_1)^2 + (c_2^k - c_2)^2] \text{ ----- (1)}$$

To modify the weight w_{11} : **This is a typical loss function for Regression Problem. (MSE)**

$$\frac{\partial E}{\partial w_{11}} = \left(\frac{\partial E}{\partial c_1} \right) \left(\frac{\partial c_1}{\partial w_{11}} \right) \text{ ----- (2)}$$

Note that w_{11} is not connected to c_2

From (1),

$$\frac{\partial E}{\partial c_1} = -(c_1^k - c_1)$$

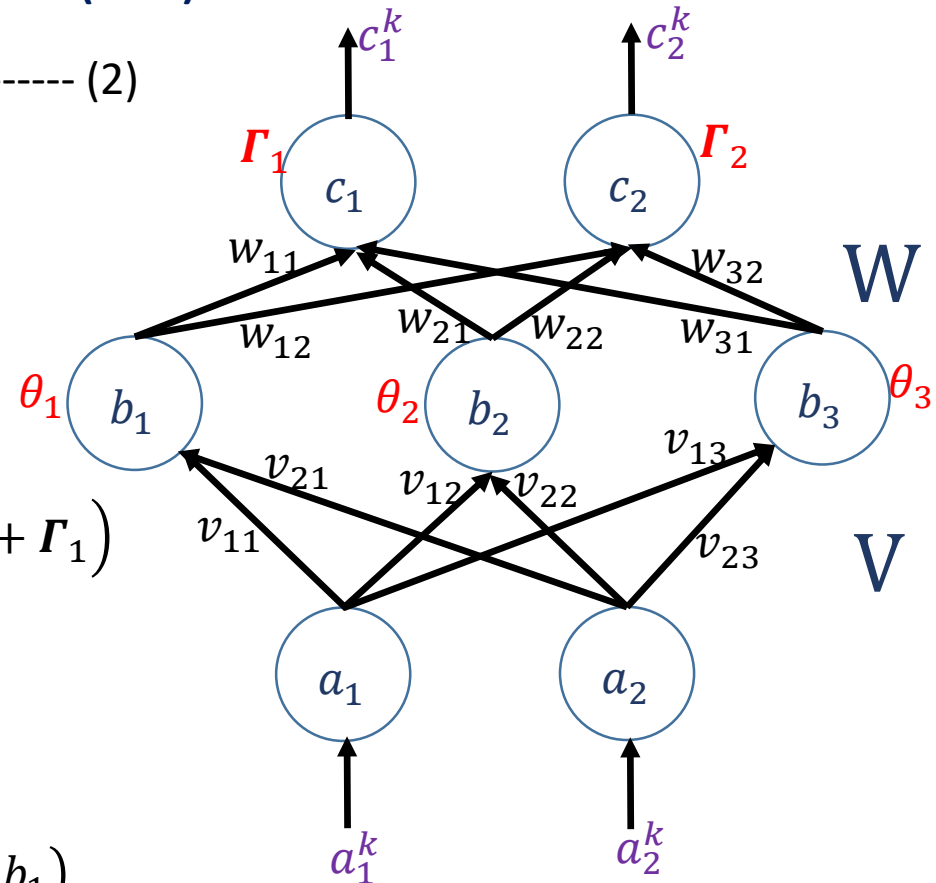
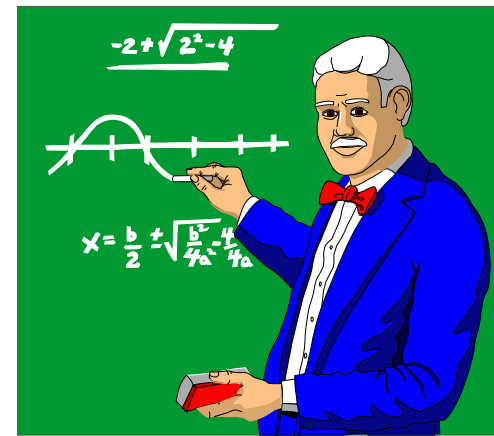
$$c_1 = f(b_1 w_{11} + b_2 w_{21} + b_3 w_{31} + \Gamma_1)$$

Therefore,

$$\frac{\partial c_1}{\partial w_{11}} = c_1(1 - c_1) b_1$$

Hence,

$$\frac{\partial E}{\partial w_{11}} = -(c_1^k - c_1)(c_1(1 - c_1) b_1)$$



$$w_{11}^{new} = w_{11}^{old} - \alpha \frac{\partial E}{\partial w_{11}} \quad (\text{Gradient Descent})$$

$$w_{11}^{new} = w_{11}^{old} + \alpha (c_1^k - c_1) (c_1 (1 - c_1) b_1)$$

Let

$$d_1 = (c_1^k - c_1) (c_1 (1 - c_1)) \quad \text{----- (4)}$$

Hence,

$$w_{11}^{new} = w_{11}^{old} + \alpha b_1 d_1 \quad \text{----- (5)}$$

To modify the weight w_{12} :

$$\frac{\partial E}{\partial w_{12}} = \left(\frac{\partial E}{\partial c_2} \right) \left(\frac{\partial c_2}{\partial w_{12}} \right)$$

Note that w_{12} is not connected to c_1

From (1),

$$\frac{\partial E}{\partial c_2} = -(c_2^k - c_2)$$

$$c_2 = f(b_1 w_{12} + b_2 w_{22} + b_3 w_{32} + \Gamma_2)$$

Therefore,

$$\frac{\partial c_2}{\partial w_{12}} = c_2 (1 - c_2) b_1$$

Hence,

$$\frac{\partial E}{\partial w_{12}} = -(\mathbf{c}_2^k - \mathbf{c}_2)(\mathbf{c}_2(1 - \mathbf{c}_2) b_1)$$

But $w_{12}^{new} = w_{12}^{old} - \alpha \frac{\partial E}{\partial w_{12}}$ (Gradient Descent)

hence $w_{12}^{new} = w_{12}^{old} + \alpha(\mathbf{c}_2^k - \mathbf{c}_2)(\mathbf{c}_2(1 - \mathbf{c}_2) b_1)$

Let

$$d_2 = (\mathbf{c}_2^k - \mathbf{c}_2)(\mathbf{c}_2(1 - \mathbf{c}_2))$$

Therefore,

$$w_{12}^{new} = w_{12}^{old} + \alpha b_1 d_2$$

Similar proof is obtained for other w_{ij} .

Next, to modify the bias term Γ_i :

$$\begin{aligned} \frac{\partial E}{\partial \Gamma_1} &= \left(\frac{\partial E}{\partial c_1} \right) \left(\frac{\partial c_1}{\partial \Gamma_1} \right) \\ &= -(\mathbf{c}_1^k - \mathbf{c}_1)(\mathbf{c}_1(1 - \mathbf{c}_1)) \\ &= -d_1 \end{aligned}$$

$$\Gamma_1^{new} = \Gamma_1^{old} - \alpha \frac{\partial E}{\partial \Gamma_1}$$

$$\Gamma_1^{new} = \Gamma_1^{old} + \alpha d_1$$

Similarly, $\Gamma_2^{new} = \Gamma_2^{old} + \alpha d_2$

To modify the weight v_{11} :

$$\frac{\partial E}{\partial v_{11}} = \left(\frac{\partial E}{\partial b_1} \right) \left(\frac{\partial b_1}{\partial v_{11}} \right)$$

Note that v_{11} is not directly connected to c_1 and c_2

$$\begin{aligned} \frac{\partial E}{\partial b_1} &= \frac{\partial E}{\partial c_1} \cdot \frac{\partial c_1}{\partial b_1} + \frac{\partial E}{\partial c_2} \cdot \frac{\partial c_2}{\partial b_1} \\ &= -\left(c_1^k - c_1\right)\left(c_1(1 - c_1) w_{11}\right) - \left(c_2^k - c_2\right)\left(c_2(1 - c_2) w_{12}\right) \\ &= -d_1 w_{11} - d_2 w_{12} \end{aligned}$$

Now,

$$\begin{aligned} b_1 &= f(a_1 v_{11} + a_2 v_{21} + \theta_1), \\ \frac{\partial b_1}{\partial v_{11}} &= b_1(1 - b_1)a_1 \end{aligned}$$

Hence,

$$\frac{\partial E}{\partial v_{11}} = (-d_1 w_{11} - d_2 w_{12})b_1(1 - b_1)a_1$$

Let

$$\begin{aligned} e_1 &= b_1(1 - b_1)[w_{11} d_1 + w_{12} d_2] \\ \frac{\partial E}{\partial v_{11}} &= -a_1 e_1 \end{aligned}$$

$$v_{11}^{new} = v_{11}^{old} - \beta \frac{\partial E}{\partial v_{11}} = v_{11}^{old} + \beta a_1 e_1$$

Similar proof is
obtained for other v_{ij}

Finally, to adjust the bias θ_1 :

$$\frac{\partial E}{\partial \theta_1} = \left(\frac{\partial E}{\partial b_1} \right) \left(\frac{\partial b_1}{\partial \theta_1} \right)$$

$$b_1 = f(a_1 v_{11} + a_2 v_{21} + \theta_1),$$
$$\frac{\partial b_1}{\partial \theta_1} = b_1(1 - b_1)$$

Hence,

$$\frac{\partial E}{\partial \theta_1} = (-d_1 w_{11} - d_2 w_{12}) b_1(1 - b_1)$$

Let

$$e_1 = b_1(1 - b_1) [w_{11} d_1 + w_{12} d_2]$$

$$\frac{\partial E}{\partial \theta_1} = -e_1$$

$$\theta_1^{new} = \theta_1^{old} - \beta \frac{\partial E}{\partial \theta_1}$$

$$\theta_1^{new} = \theta_1^{old} + \beta e_1$$

Similar proof is
obtained for other θ_i